Ders 9 Genişletilmiş Notu: Yığın Uygulamaları, Infix-Prefix Dönüşümü ve Kuyruk Yapıları

Ders 9 Genişletilmiş Notu: Yığın Uygulamaları, Infix-Prefix Dönüşümü ve Kuyruk Yapıları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Yığının Kullanım Alanları, Avantajları ve Dezavantajları
- 2. Infix İfadeyi Prefix İfadeye Dönüştürme
- 3. Ekstra Alan Kullanmadan Yığını Ters Çevirme
- 4. Kuyruk Yapısı ve FIFO Mantığı
- 5. Kuyruğun Bağlı Liste ile Gerçeklenmesi
- 6. Dizi ile Normal Kuyruk ve Başlangıç Değeri Sorunu
- 7. Dairesel Kuyruk
- 8. Yığın ile Kuyruğun Karşılaştırılması
- 9. İki Yığınla Kuyruk Gerçekleme
- 10. Kuyruk ile Yığın Gerçekleme
- Örnek Sorular
- Dersin Toparlaması

1. Yığının Kullanım Alanları, Avantajları ve Dezavantajları

Geçen hafta yığın (stack) veri yapısının temel metotları ve yığınla çözülebilen problemler konuşulmuştu. Hoca bu hafta önce hızlı bir toparlama yaptı, sonra yığınla çözülen birkaç yeni probleme geçti. Dersin ikinci yarısı kuyruk (queue) yapısına ayrıldı.

1.1 Yığın Nerede Karşımıza Çıkar?

Yığın, son giren ilk çıkar (last in first out, LIFO) ya da ilk giren son çıkar (first in last out, FILO) mantığıyla çalışır. Yani gelen sırayla hizmet verilmez, gelen sıranın tam tersi yönde hizmet verilir. Hoca bu yüzden bu yapıya uyan problemlerin yığınla kurgulanması gerektiğini söyledi.

Hocanın saydığı kullanım alanları şunlardır:

- **Fonksiyon çağrısı ve özyineleme (function call, recursion):** Belleğin belli bir alanı yığın bölgesi (stack) olarak kullanılır ve bu alanın bir sınırı vardır. Çok fazla iç içe fonksiyon çağrılırsa bir süre sonra bu alan dolar. Dolduğunda yığın taşması (stack overflow) hatası alınır ve program sonlanır (terminate). “Stack overflow” ifadesi de buradan gelir.
- **Geri al / yinele (undo / redo):** En son yapılan işlem geri alınır ya da silinen bir kelime yerine konur. En son yapılan iş en üstte durduğu için yığın en uygun yapıdır.
- **İfade değerlendirme (expression evaluation):** Aritmetik ifadelerde yığın kullanılır. Geçen hafta infix’ten postfix’e dönüşüm ve bir derleyicinin bu ifadeyi `push` ve `pop` işlemleriyle adım adım nasıl değerlendirdiği görülmüştü.
- **Tarayıcı geçmişi (browser history):** En son erişilen adres en günceldir ve yığının tepesinde (`top`) durur.

- **Parantez dengesi:** Parantezlerin ve benzer ikili sözdizimi öğelerinin dengeli ve doğru sırayla kullanılıp kullanılmadığını, insanın gözle yaptığı kontrolü bilgisayarın analitik olarak yapabilmesi için yığın tercih edilir.
- **Geri izleme (backtracking):** Hoca bunu ilerleyen yıllarda göreceğimizi söyledi. Labirent çözümünde gidilen yol çıkmaz sokağa varınca bir adım geri dönülür ve yeni bir yol denenir. En son kalınan nokta yığının tepesindedir. Oradan da başarı olmazsa daha önceki noktaya dönülür. Bu mantıkla birçok labirent problemi kolayca formüle edilir.

Gerçek hayattan örnekler de verildi: kitaplıkta en alta konan kitaba en son erişilmesi, bulaşık yıkanırken tabakların üst üste dizilmesi. Bilgisayar tarafında metin editörleri (undo/redo), web tarayıcıları, arama kaydı (call log), e-postalar, telefondaki resimler, bildirimler (notification) ve indirme geçmişi (download history) sayıldı. Hepsinde en güncel veri en üstte, eski veri en altta durur. Yığın bu nedenle soyut veri tipi (abstract data type) olarak en sık kullanılan yapılardan biridir.

1.2 Avantajlar ve Dezavantajlar

Avantajlar:

- Gerçeklemesi kolaydır.
- Veriye `push` ve `pop` ile $O(1)$ zamanda ulaşılır.
- Fonksiyon çağrısı mimarisine birebir oturur, özyinelemeli fonksiyonların çağrılmasını kolaylaştırır.
- Geri izleme problemlerini rahat çözülebilir hale getirir.
- Derleyici tasarımında (compiler design) ve undo/redo işlemlerinde çok kullanılır.

Dezavantajlar:

- Aradan veri çekilemez. Rastgele erişim (random access) yoktur.
- Taşma (overflow) ve boşalma (underflow) sorunları vardır. Yığın dolabilir, bunun iyi yönetilmesi gerekir. Yığının içinde hiçbir şey kalmayabilir; hoca bu durumda yığının verimli kullanılmadığını söyledi.
- Bazı sıralama (sorting) ve arama (searching) yaklaşımları için yığın kullanmak makul değildir. Yığın daha çok ekleme ve silme (insert, delete) işlemlerinin ağır bastığı yerlerde kullanılır.
- Dinamik bellek ayırmada (dynamic memory allocation) kullanılan heap bölgesi, yığın bölgesinden çok daha geniştir. Yığın için ayrılan alanların kapasitesi sınırlıdır. Problemler bu göz önünde tutularak formüle edilmelidir.

Hoca bu dersin yığın problemlerini şöyle sıraladı: bir elemandan sonra gelen ilk büyük elemanı bulma (next greater element), infix'ten prefix'e dönüşüm ve bir yığını ek alan kullanmadan ters çevirme. Derste ikinci ve üçüncü problem işlendi.

2. Infix İfadeyi Prefix İfadeye Dönüştürme

Geçen hafta infix'ten postfix'e dönüşüm görülmüştü. Hoca bu notasyonlar arasındaki bütün dönüşümlerin dikkate değer olduğunu söyledi ve infix'ten prefix'e dönüşümle başladı.

2.1 Isınma Sorusu: $A * B + C/D$

Hoca tahtaya $A * B + C/D$ ifadesini yazdı ve bunun prefix karşılığını sordu. Öğrencilerden gelen cevap şudur:

```
+ * A B / C D
```

Hoca cevabı birlikte kontrol etti. Önce $A * B$ yapılmalı, sonra C/D, en son da bu iki sonuç toplanmalıdır. Prefix ifadede de önce `* A B`, sonra `/ C D` yazılmış, en başa da bu iki sonucu toplayan `+` konmuştur. Cevap doğrudur.

2.2 Üç Adımlı Algoritma

Bu dönüşüm gözle yapıldı. Hoca bunu bir yığınla, kaba bir algoritma olarak nasıl kurabileceğimizi sordu ve daha küçük bir örnek verdi: $A + B * C$. Bu ifadenin prefix karşılığı `+ A * B`

C'dir.

Hoca ipucu olarak infix ifadeyi tersten yazdı: $C * B + A$. Bir öğrenci "en yüksek öncelikli işlem yığının en üstünde olmalı" dedi. Hoca bunun doğru olduğunu, ama burada istenenin başka bir şey olduğunu söyledi: Tersten yazılan ifade hâlâ bir infix ifadedir. Bu ifade postfix'e dönüştürülür. Sonra bir öğrencinin önerisiyle sonuç yeniden ters çevrilir.

Adım	İşlem	Sonuç
1	Infix ifadeyi ters çevir (reverse)	`C * B + A`
2	Infix'ten postfix'e dönüştür	`C B * A +`
3	Sonucu yeniden ters çevir	`+ A * B C`

Böylece üç temel adımda infix'ten prefix'e dönüşüm yapılmış olur. Hoca, prefix'ten infix'e, postfix'ten infix'e ya da prefix'ten postfix'e gibi diğer dönüşümlerin de aynı bakış açısıyla, ifadeyi ters yüz edip belli kurallar çerçevesinde yapılabildiğini söyledi.

Soru: İlk başta ifadeyi tersine çevirmek neden gerekli? Bu sadece bu örneğe, yani çarpmanın öncelikli olmasına özel bir şey mi?

Cevap: Ters çevirince sonradan yapılması gereken işlemler öne alınır. Postfix dönüşümü araya bir dönüştürücü gibi girer. Postfix'e çevrilen ters ifade, olması gereken yapının tam tersi olur; son adımdaki ters çevirme onu doğru sıraya getirir. Bu örneğe özel değildir: Verilen her infix ifade için önce ters çevirme, sonra postfix'e dönüştürme, sonra yeniden ters çevirme yapılır. Önce postfix'e çevirip sonra ters çevirirseniz doğru prefix ifadeyi elde edemezsiniz; hoca bunu denemeyi önerdi.

Hocanın önerdiği deneme python ile yapıldı. $A + B * C$ önce postfix'e çevrilirse `A B C * +` olur. Bunun tersi `+ * C B A`'dır. Doğru prefix ise `+ A * B C`'dir. İkisi aynı değildir. Isınma örneğinde de `A B * C D / +` ters çevrilince `+ / D C * B A` çıkar, doğrusu `+ * A B / C D`'dir.

2.3 Parantezler ve Öncelik Karşılaştırmasındaki Fark

Hoca parantezli bir örnek yazdı ve bir önceki örneğe göre farkın ne olacağını sordu. Bir öğrenci cevapladı: İfade ters çevrildikten sonra parantezler de kendi içinde ters çevrilmelidir. Yani kapa parantez aç parantez, aç parantez de kapa parantez yapılır. Hoca bunu onayladı.

Hoca bir küçük ayrıntı daha verdi. Infix'ten postfix'e dönüşümde, gelen operatöre göre yığındaki operatörün önceliği "eşit ya da büyük" ise bir işlem (yığından çıkarma) yapılıyordu. Prefix dönüşümünde, ters çevrilmiş ifadeyle çalışırken yalnız "büyük" olma durumu gözetilir. Hoca bütün ayrıntıyı vermediğini, amacının bakış açısını göstermek olduğunu söyledi ve infix'ten prefix'e akışın araştırılıp incelenmesini önerdi.

> **Ek bilgi (derste anlatılmadı):** "Yalnız büyük" kuralının nedeni soldan birleşmedir (left associativity). $A - B - C$ ifadesi $(A - B) - C$ demektir ve prefix karşılığı `- - A B C`'dir. Ters çevrilmiş `C - B - A` ifadesinde eşit öncelikte de yığından çıkarma yapılırsa sonuç `- A - B C` olur; bu $A - (B - C)$ demektir ve yanlıştır. Sağdan birleşen üs alma (`^`) için durum tersine döner; ters ifadede eşit öncelikte de çıkarma yapılır. Bu iki sonuç python ile kontrol edildi.

Hoca infix'ten prefix'e dönüşümün infix'ten postfix'e dönüşümden daha zahmetli olduğunu söyledi. Bu yüzden derleyici yapıları genelde postfix üzerinden kurgulanır ve öyle çalıştırılır. Prefix biraz daha maliyetli bir durum oluşturur.

2.4 Karışık Örnek: Infix, Prefix ve Postfix

Hoca antrenman için üç sütunlu bir tablo açtı: infix, prefix, postfix. Karışık bir örnekle başladı:

$$A / B \wedge C + D * E - A * C$$

Şapka işareti (`^`) üs almadır. Parantezden sonra en yüksek öncelik ondadır. İşlem sırası şöyledir:

- 1. $B \wedge C$ hesaplanır.
- 2. A bu sonuca bölünür: $A/(B \wedge C)$.
- 3. $D * E$ hesaplanır.
- 4. 2. ve 3. adımın sonuçları toplanır.
- 5. $A * C$ hesaplanır.
- 6. 4. adımın sonucundan $A * C$ çıkarılır.

Prefix ifadede her operatör, işlenenlerinin önüne yazılır. Hoca önce `^ B C`'yi koydu, sonra bölmeyi, çarpmaları, toplamayı ve en dışa çıkarmayı yerleştirdi. Postfix ifadede ise operatör işlenenlerinin arkasına yazılır. Hoca postfix'i yazarken bir yerde düzeltme yaptı ve adımları yeniden sayarak tamamladı.

Gösterim	İfade
Infix	`A / B ^ C + D * E - A * C`
Prefix	`- + / A ^ B C * D E * A C`
Postfix	`A B C ^ / D E * + A C * -`

İki sonuç python ile kontrol edildi: Her iki ifade de rastgele değerlerde infix ifadeyle aynı sonucu verir.

2.5 Alıştırma İfadeleri

Hoca iki ifade daha yazdı ve bunların prefix ve postfix karşılıklarının çıkarılmasını istedi. Sistematiğin oturması için bu tür birkaç örnek yapmanın faydalı olacağını söyledi. Çözümler derste verilmedi; aşağıdaki "Örnek Sorular" bölümünde, python ile kesinleştirilmiş olarak yer alıyor.

- $A \wedge B * C - D + E/F/(G + H)$
- $A - B/(C * D \wedge E)$

3. Ekstra Alan Kullanmadan Yığını Ters Çevirme

3.1 Problem

İçinde alttan üste 1, 2, 3, 4 olan bir yığın verilmiş olsun. Bunu ters çevirmek istiyoruz. Geçen hafta gösterilen yaklaşımda ikinci bir yığın kullanılıyordu: İlk yığından `pop` edilen eleman ikinci yığına `push` edilir. Sonuçta ikinci yığında elemanlar 4, 3, 2, 1 sırasıyla durur. Bu yöntem n elemanlık ek alan (extra space) ister.

> **Ders sonrası not:** Hoca, n elemanlık ek alanla bu işin "O(1) karmaşıklıkla" yapıldığını söyledi. Doğrusu $O(n)$ zaman ve $O(n)$ ek alandır; her eleman bir kez `pop` ve bir kez `push` edilir.

Hoca şimdi ek alan kullanmadan bunun nasıl yapılacağını sordu. Her şey serbestti, farklı veri yapıları da kullanılabilirdi.

3.2 Bir Öneri ve Hocanın Cevabı

Bir öğrenci, dizi ile gerçeklenen yığında `top` ve kapasiteyi (`max`) tuttuklarını, `top`u sıfırdan başlatıp ilerletince elemanların baştan çekilebileceğini önerdi. Hoca bunun teknik olarak yapılabildiğini, ama yığın veri yapısında yığının en altındaki bilgiye sahip olmadığımızı söyledi. Yalnız `top`, yani yığın işaretçisi (stack pointer) üzerinden en üstteki elemana erişme hakkımız vardır. Dizi ile kurulmuş olsa bile alt taraflara erişilmez. Bu kısıtla farklı bir çözüm aranmalıdır.

3.3 Bağlı Liste ile Çözüm

Bir başka öğrenci çift yönlü bağlı liste (doubly linked list) kullanıp bağlantıları değiştirmeyi önerdi. Hoca anahtar noktanın bağlı liste yapısından faydalanmak olduğunu söyledi. Yığın dizi ile de, bağlı liste ile de gerçeklenebilir.

Bağlı liste ile kurulan yığında eleman eklenme sırası şöyledir: İlk eleman 1 gelir ve `head` onu gösterir. Sonra 2 gelir; en tepeye 2 geçer ve 2, 1'e bağlanır. Sonra 3, sonra 4 gelir. Sonuçta `head` 4'ü gösterir ve liste `4 -> 3 -> 2 -> 1 -> NULL` biçimindedir.

Ters çevirme için liste `NULL` görülene kadar dolaşılır. Her adımda bir düğümün bağlantısı ters yöne çevrilir. Hoca bunu şöyle anlattı: 4'ün bağlantısı artık 3'ü değil, ters yöndeki düğümü gösterir; 3'ün bağlantısı 4'ü, 2'ninki 3'ü, 1'inki 2'yi gösterecek biçimde çevrilir. En sonda `head`, eskiden en altta olan ve bağlantısı `NULL` olan düğümün (1) adresine atanır. Artık `head` 4'ü değil 1'i gösterir. Böylece hiç ek alan kullanılmadan yapı ters çevrilmiş olur.

> **Ders sonrası not:** Hoca anlatırken "previous" ve "next" sözcüklerini karışık kullandı. Tek yönlü bağlı listede standart yöntem üç işaretçi (`prev`, `cur`, `next`) ile her düğümün `next` alanını bir önceki düğüme çevirmektir. Aşağıdaki kod `gcc -Wall` ile derlenip çalıştırıldı.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int data;
    struct node *next;
} NODE;

void push(NODE **head, int value) {
    NODE *n = malloc(sizeof(NODE));
    if (n == NULL) exit(1);
    n->data = value;
    n->next = *head;          /* yeni dugum en uste */
    *head = n;
}

void reverseStack(NODE **head) {
    NODE *prev = NULL, *cur = *head, *nxt;
    while (cur != NULL) {     /* NULL gorulene kadar ilerle */
        nxt = cur->next;      /* sonraki dugumu kaybetme */
        cur->next = prev;     /* bagi ters cevir */
        prev = cur;
        cur = nxt;
    }
    *head = prev;             /* eski en alttaki dugum artik tepe */
}

void print(NODE *h) {
    printf("tepe -> ");
    for (; h != NULL; h = h->next) printf("%d ", h->data);
    printf("-> NULL\n");
}

int main(void) {
    NODE *head = NULL, *t;
    int i;
    for (i = 1; i <= 4; i++) push(&head, i);
    print(head);
    reverseStack(&head);
    print(head);
    while (head != NULL) { t = head; head = head->next; free(t); }
    return 0;
}
```

Çıktı:

```
tepe -> 4 3 2 1 -> NULL
```

```
tepe -> 1 2 3 4 -> NULL
```

Ters çevirme $O(n)$ zamanda ve $O(1)$ ek alanla yapılır.

Hoca buradan genel bir ders çıkardı: İhtiyaca ve eldeki kaynağa göre farklı veri yapıları kullanarak bazen alan karmaşıklığını (space complexity) düşürüp zaman karmaşıklığını (time complexity) yükseltebilirsiniz. Bazen de tam tersine, ek alan kullanarak zamandan kazanırsınız. Hoca, verilen bir bağlı listenin ek alan kullanmadan ters çevrilmesini alıştırma olarak önerdi ve takılan noktaların kendisine iletilebileceğini söyledi.

4. Kuyruk Yapısı ve FIFO Mantığı

4.1 Tanım ve Örnekler

Kuyruk (queue) da bir soyut veri yapısıdır. Temel mantığı ilk giren ilk çıkar (first in first out, FIFO) mantığıdır; yığının tam tersidir. İlk gelen ilk hizmeti alır.

Hocanın örnekleri:

- Otobüs beklerken sıraya ilk giren kişi otobüse ilk binen kişidir. Hoca günlük hayatta farklı manzaralarla karşılaşılabildiğini, ama normalde beklenenin bu olduğunu söyledi.
- Bankada ilk gidenin ilk hizmeti alması beklenir. Ama orada öncelikli kuyruklar da vardır: 65 yaş üstü ya da bankanın öncelikli müşterileri için ayrı mekanizmalar. Bu konu ilerleyen derslerde konuşulacak.
- Tek bir kaynağı birden fazla kişi kullanıyorsa sıraya girilir. Ortak kullanılan bir yazıcıya farklı bilgisayarlardan baskı isteği gelirse istekler sırayla basılır. Sizden önceki kişinin elli sayfası varsa o elli sayfanın çıkmasını beklersiniz.
- Yönlendirici (router) ve anahtar (switch), gelen ağ paketlerini geliş sırasına göre iletir (forward).
- İşletim sisteminde süreçler (process) geldiği sırayla hizmet alır.

Kaynakların talep edenler arasında hakkaniyetli paylaşılması için FIFO mantığı günlük hayatta ve bilgisayar biliminde çok sık kullanılır.

4.2 `front` ve `rear` İşaretçileri

Yığından farklı olarak kuyrukta tek bir `top` işaretçisi yoktur. Kuyruğun başını ve sonunu gösteren iki işaretçi vardır: `front` ve `rear`.

- `front`, bir sonraki işleme girecek, kaynağı sıradaki kullanacak elemanı gösterir.
- `rear`, kuyruğa en son katılan elemanı ve bir sonraki gelecek elemanın nereye yerleşeceğini takip etmeye yarar.

Örneğin kuyruğun başında A varsa, arkasından B, C ve D gelmiştir. `front` A'yı gösterir.

4.3 Kuyruk Türleri

- **Standart kuyruk:** Şu ana kadar konuşulan yapı.
- **Dairesel kuyruk (circular queue):** Dairesel bağlı listede olduğu gibi alanın daha verimli kullanılmasını sağlar. Kuyruktan eleman ayrıldıkça yeni gelen elemanlar onun yerini alabilir.
- **Öncelikli kuyruk (priority queue):** Bankadaki gibi, önceliği en yüksek olan bekleyen eleman kuyruğun başına yerleşir. Genellikle yığın ağacı (heap data structure) ile gerçeklenir.
- **Çift uçlu kuyruk (double ended queue):** İlerleyen derslerde konuşulacak.

4.4 Dizi Üzerinde `enqueue` ve `dequeue`

Yığındaki `push` ve `pop` işlemlerinin kuyruktaki karşılıkları `enqueue` ve `dequeue`'dur. `enqueue` bir elemanın kuyruğa eklenmesi, `dequeue` bir elemanın kuyruktan hizmet alıp çıkmasıdır.

Hoca slayttaki boş bir dizi üzerinden akışı anlattı:

- Kuyruk boşken `front` ve `rear` aynı yeri, en baş noktayı gösterir. Bu değer gerçeklemeye göre 0 da olabilir, −1 de.
- `enqueue(A)` ile A ilk göze yerleşir. `front` hâlâ orayı gösterir, `rear` artık bir sonraki gözü gösterir.
- Sonraki elemanlar `rear`ın gösterdiği yere yerleşir. B, C, D eklenince `rear` bir sonraki boş gözü gösterir. Henüz eleman çıkmadığı için `front` hâlâ A'dadır.
- `dequeue` yapılınca ilk çıkan A'dır. "Boşalma" demek o gözün silinmesi değildir; orada hâlâ A yazıyor olabilir. Önemli olan `front`un artık B'yi göstermesidir.
- Üç `dequeue` daha yapılırsa B, C ve D de çıkar. `front` ve `rear` yine aynı boş gözü gösterecek biçimde konumlanır.

4.5 Elemanları Kaydırmak Neden Tercih Edilmez?

Bir başka yaklaşımda A çıkarıldıktan sonra B, C, D kuyruğun başına kaydırılır. Hoca bunun her `dequeue` sonrası kuyruğun yeniden düzenlenmesi demek olduğunu, ekstra bir iş ve yük getirdiğini söyledi. Bağlı listeyi ekleme ve silmede diziye tercih etmemizin sebebi de buydu: Bir eleman çıktığında ya da geldiğinde bütün elemanların yerini değiştirmemek. Bu yüzden kuyruk genelde işaretçiler ve dairesel kuyruk üzerinden gerçeklenir.

5. Kuyruğun Bağlı Liste ile Gerçeklenmesi

Kuyruk da yığın gibi hem dizi hem bağlı liste ile gerçeklenebilir. Bağlı listede `front` işaretçisi listenin başlangıç noktasını tutar. Daha önce bağlı listelerde listenin son düğümünü bir `tail` işaretçisiyle tutabileceğimiz söylenmişti. Kuyrukta da son düğümün adresi `rear` işaretçisinde tutulur. Böylece kuyruk bağlı liste üzerinde kolayca gerçeklenir.

- **`dequeue`:** Baştaki eleman (`front`un gösterdiği düğüm, örneğin A) alınır. `front` artık B'yi göstermeye başlar.
- **`enqueue`:** `rear` son düğümü (örneğin F) gösterirken yeni düğüm yaratılır, son düğüme bağlanır ve `rear` artık yeni düğümün adresini gösterir.

6. Dizi ile Normal Kuyruk ve Başlangıç Değeri Sorunu

6.1 Kuyruk İşlemleri

Kuyrukla ilgili temel işlemler şunlardır:

- **`isEmpty`:** Kuyruk boş mu? Boşsa `dequeue` yapılmamalı; hata üretilmeli ya da kullanıcı bilgilendirilmelidir.
- **`isFull`:** Kuyruk dolu mu? Doluysa `enqueue` yapılmamalı; yine hata üretilmeli ya da kullanıcı bilgilendirilmelidir.
- **`enqueue`:** Bir eleman gönderilir, kuyruğa yerleştirilir.
- **`dequeue`:** Kuyruktan bir eleman alınır.
- **`initQueue`:** Kuyruğu ilklendirir.

6.2 Derste Gösterilen Kod

Hoca slaytta iki farklı gerçeklemenin `isEmpty` ve `isFull` fonksiyonlarını gösterdi. Birinde `size == 0` ise kuyruk boştur. Diğerinde `front == -1` ise kuyruk boştur. Doluluk için birinde `size`, diğerinde `rear` kontrol edilir: `rear`, `MAX - 1`'i gösteriyorsa dizinin sonuna gelinmiştir ve kuyruk doludur. Bu kısım dairesel değil, normal kuyruk içindir.

`enqueue` ve `dequeue` fonksiyonlarının özü şöyledir:

```
int isEmpty() { return front == -1; }
int isFull()  { return rear == MAX - 1; }

int enqueue(int value) {
    if (isFull()) return 0;      /* eklenemedi */
    q[++rear] = value;           /* once rear artar, sonra yerlestirilir */
    return 1;
```

```
}

int dequeue(int *x) {
    if (isEmpty()) return 0;      /* kuyruk bos */
    *x = q[front++];              /* once deger alinir, sonra front artar */
    return 1;
}
```

- `enqueue`'da `isFull` 1 döndürürse fonksiyon 0 döndürür; bu, elemanın eklenemediği anlamına gelir. `isFull` 0 döndürürse `rear` bir artırılır ve değer `rear`ın gösterdiği yere yerleştirilir. Sonra 1 döndürülür.
- Hoca burada `rear`ı önce artırıp sonra yerleştirdiklerini, implementasyonun buna göre yapıldığını, farklı biçimde de yapılabileceğini söyledi.
- `dequeue`'da alınan değer `x`'in adresine yazılır. Dönüş değeri işin başarıyla yapılıp yapılmadığını gösterir. Kuyruk boşsa 0 döner.
- `front++` ifadesinde artırma operatörü sonra geldiği için önce `front`un değeri kullanılır, sonra `front` bir artırılır. `++front` olsaydı önce artırılır, sonra kullanılırdı.

Soru: Kuyruk için de bir `struct` tanımlamamız gerekiyor mu?

Cevap: Kuyruk için bir `struct` yapısı oluşturulabilir. Burada dizi ve global değişkenlerle gerçeklendiği için buna gerek duyulmadı. Bağlı liste kullanılıyorsa bir yapı (structure) tanımlamak gerekir.

6.3 Hocanın Gösterdiği Sorun

Hoca kuyruk boşken `front` ve `rear`ın ikisinin de -1 olduğunu kabul etti ve `enqueue(A)` sonrası bu iki işaretçinin nereyi göstereceğini sordu. Tartışmada şu ortaya çıktı:

- Koda göre `rear` artırılıp yerleştirildiği için `rear = 0` olur.
- `enqueue` içinde `front`la ilgili hiçbir işlem yapılmadığı için `front` -1 olarak kalır.
- Bu durumda `dequeue` çağrılırsa `isEmpty`, `front == -1` olduğu için kuyruğu boş sanar ve A çıkarılamaz.
- `isEmpty` kontrolü `rear` üzerinden yapılsa bile `front` -1 olduğu için `q[-1]` gözünden eleman çekilmeye çalışılır. Oysa çekilmesi gereken `q[0]`'dır.

Hoca bu doğrultuda implementasyonun nasıl yapıldığının ve kontrollerin nasıl kurulduğunun çok önemli olduğunu vurguladı.

Derste konuşulan ilk öneriler şunlardır:

- `dequeue`'da `front++` yerine `++front` yazmak.
- `enqueue` sırasında, `rear` artırılırken `front`u da değiştirmek ki -1'de kalmasın.

Hoca ikisini birlikte düşünüp devam etti. İlk eleman eklenirken `front` 0'a çekilmiş olsun; `front` ve `rear` 0'ı gösterir. Sonra B eklenir; `rear` 1'i gösterir. Şimdi `dequeue` yapılır ve `++front` kullanılır. `front` 1'e gelir ve B çıkarılır. Oysa çıkması gereken A'dır. Hoca "Demek ki bu bize bir çözüm olmuyor" dedi ve kodu eski haline döndürdü. Sorunun özü, `front`un başlangıç değeri ile artırma sırasının birbiriyle uyumlu olmamasıdır.

Hoca öğrencilere beş dakika verip `enqueue` ve `dequeue` sırasında bir sorun olup olmadığını, başlangıç noktalarının nerede olması gerektiğini kâğıt kalemle değerlendirmelerini istedi.

6.4 Önerilen Çözümler

Süre sonunda üç fikir konuşuldu:

- **Bir öğrencinin önerisi:** `front` ve `rear` 0'dan başlar. `enqueue`'da önce `q[rear] = value` yapılır, sonra `rear++`. `rear == 0` ise kuyruk boştur. Dolu kontrolü `rear == MAX - 1` yerine `rear == MAX` olur; son eleman eklenince `rear` dizinin dışını gösterir. `dequeue` değişmez, `front` yalnız 0'dan başlar. Öğrenci, kâğıt üzerinde çalıştığını ama denenmesi gerektiğini söyledi.
- **İkinci öneri:** `front` ve `rear` yine -1'den başlar. `isEmpty` kontrolü `front == rear` olur. Başta ikisi de -1 olduğu için kuyruk boş görünür. A eklenince `rear = 0` olur, `front` -1'de

kalır ve kuyruk boş sayılmaz. `dequeue`'da `++front` kullanılır; `front` 0'a gelir ve A çıkar. Hoca bunu onayladı.

- **Hocanın gösterdiği `size` yaklaşımı:** Ayrı bir `size` sayacı tutulur. `size == 0` ise kuyruk boştur. Her `enqueue`'da `size` bir artırılır, her `dequeue`'da bir azaltılır. Kuyruk tamamen boşaldığında `front` ve `rear` mutlaka başlangıç konumlarına geri alınmalıdır.

Hoca bu fikirlerin hepsinin denenip kontrol edildikten sonra kullanılabilecek fikirler olduğunu söyledi.

> **Ders sonrası not:** Hoca `size` tutan sürümde doluluk koşulunu `size == MAX - 1` olarak söyledi. `size` eleman sayısını tuttuğu için doğru koşul `size == MAX`'tır. Ayrıca hoca bir yerde `size`'ın "eleman ekledikten sonra" azaltıldığını söyledi; doğrusu eleman çıkarıldıktan sonra azaltılmasıdır.

> **Ders sonrası not:** Önerilerin dördü C ile derlenip denendi. İkinci öneri (`front = rear = -1`, `q[++rear]`, `q[++front]`, boşluk için `front == rear`) A ve B'yi doğru sırayla çıkarır ve boş kuyrukta `dequeue`'yu reddeder. `size` sürümü de doğru çalışır. Birinci öneride ise `rear == 0` boşluk kontrolü yalnız başlangıcı yakalar: A eklenip çıkarıldıktan sonra `front = 1`, `rear = 1` olur ve boş kuyruktan yeniden `dequeue` yapılabilir. Bu öneride boşluk kontrolü `front == rear` olmalıdır.

Hoca bu normal kuyruk gerçeklemesinin çok sık kullanılmadığını söyledi. Kontrol `size` ve bir en büyük değer üzerinden yapılır, hiçbir dönme (rotasyon) yoktur. Bu yüzden boşu boşuna yer kaybedilir. Örneğin 4 gözlük dizide A ve B eklenip çıkarıldıktan sonra C ve D eklenirse, kuyrukta iki eleman olmasına rağmen `rear = MAX - 1` olduğu için yeni eleman eklenemez (C ile denendi). Asıl anlamlı kullanım dairesel kuyruktur.

7. Dairesel Kuyruk

7.1 Neden Dairesel?

Hoca önce slayttaki örneği anlattı: A, B, C, D `enqueue` ile yerleşir. Sonra `dequeue` ile A ayrılır. B, C, D varken E ve F eklenir. Yeni eleman G geldiğinde dairesel kuyrukta dizinin en başına, A'nın boşalttığı yere yerleşebilir. En önemli avantaj diziyi çok daha verimli kullanmaktır.

Normal kuyrukta `front` başlangıçta `rear`dan küçüktür. Dairesel kuyrukta başa dönme olduğu için `rear`, `front`tan daha küçük bir konuma gelebilir. Bu yüzden kuyruğun ne zaman dolduğunu ve ne zaman tamamen boşaldığını belirlemek daha incelikli bir çözüm ister.

Slayttaki örnekte G eklendikten sonra `rear` ile `front` yan yanadır ve yeni eleman eklenemez, çünkü kuyruk doludur. B çıkarılınca `front` C'yi gösterir, A'nın eski yeri boşalır ve aralarında yeniden bir mesafe açılır.

7.2 Tahtadaki Örnek

Hoca tahtada bir örnek daha yürüttü: A, B, C, D kuyruğa yerleşmiş olsun. `front` A'yı, `rear` D'yi gösterir.

İşlem	Kuyruk (`front`tan `rear`a)	`front`	`rear`
A, B, C, D eklendi	A B C D	0	3
`dequeue` (A çıkar)	B C D	1	3
`dequeue` (B çıkar)	C D	2	3
`enqueue(E)`	C D E	2	4
`enqueue(F)`	C D E F	2	5
`dequeue` (C çıkar)	D E F	3	5
`enqueue(G)`	D E F G	3	0

Son adımda `rear` dizinin sonunda olduğu için başa döner ve G ilk göze yerleşir. Kuyruk artık D, E, F ve G'den oluşur.

Hoca sonra `front` ve `rear`ın hangi konumda olduğunda kuyruğun dolu sayılacağını sordu. Bir öğrenci "aynı indekse gelirlerse" dedi, sonra "arkalı önlü olurlarsa" diye düzeltti. Hoca iki

koşul olduğunu söyledi: biri başlangıç koşulu, diğeri ilerleyen turlarda `rear`ın `front`un hemen arkasına gelmesi.

7.3 Fonksiyonlar

Dairesel kuyrukta `initQueue`, `front` ve `rear`ı −1 yapar. `isEmpty`, `front == -1` ise kuyruğun boş olduğunu söyler.

Kuyruğun boş olması yalnız başlangıç anına özgü değildir. Örneğin yukarıdaki kuyruktan D, E, F ve G çıkarılırsa kuyruk yine boşalır. Bu durumun yakalanabilmesi için son eleman çıkarıldığında `front` mutlaka −1'e çekilmelidir.

`isFull` iki koşuldan biri sağlanırsa doğru döner, bu yüzden aralarında "veya" vardır:

- `front == rear + 1`: `rear` dizinin başına dönmüş ve `front`un hemen arkasına gelmiştir.
- `front == 0` ve `rear == SIZE - 1`: Hiç durmadan eleman gelmiş, hiç eleman çıkmamıştır. `front` başta kalmış, `rear` sona kadar gitmiştir. Bir öğrenci bu durumu "`rear` dizinin sonunda, `front` başta; döndüğünde aslında `rear + 1` gibi" diye açıkladı. Hoca örnek olarak A'dan F'ye kadar her şeyin eklendiği, `front`un ilk gözü, `rear`ın son gözü gösterdiği durumu verdi.

`enqueue`'da önce doluluk kontrol edilir. Kuyruk dolu değilse iki iş yapılır:

- `front == -1` ise `front` 0 yapılır. Bu yalnız kuyruğa ilk kez eleman eklenirken çalışır.
- `rear = (rear + 1) % SIZE` yapılır. `rear` dizinin sonuna gitmiş olabilir. Boyuta göre mod alınınca başa döner.

Sonra eleman `q[rear]`a yerleştirilir.

`dequeue`'da önce boşluk kontrol edilir. Sonra `front`un gösterdiği eleman alınır. Ardından `front == rear` kontrolü yapılır. Eşitse kuyruktaki son eleman çıkmıştır; `front` ve `rear` yeniden −1'e çekilir. Eşit değilse `front = (front + 1) % SIZE` yapılır.

Aşağıdaki kod derste anlatılan fonksiyonlardan kuruldu ve `gcc -Wall` ile derlenip çalıştırıldı.

```c
#include <stdio.h>

#define SIZE 4

char q[SIZE];
int front, rear;

void initQueue(void) { front = -1; rear = -1; }

int isEmpty(void) { return front == -1; }

int isFull(void) {
    return (front == rear + 1) || (front == 0 && rear == SIZE - 1);
}

int enqueue(char value) {
    if (isFull()) return 0;
    if (front == -1) front = 0;          /* ilk eleman */
    rear = (rear + 1) % SIZE;
    q[rear] = value;
    return 1;
}

int dequeue(char *x) {
    if (isEmpty()) return 0;
    *x = q[front];
    if (front == rear) {                 /* son eleman da cikti: bosalt */
        front = -1;
        rear = -1;
```

```
    } else {
        front = (front + 1) % SIZE;
    }
    return 1;
}
```

7.4 Dört Gözlük Örnek

Hoca `SIZE = 4` olan bir dizide fonksiyonları adım adım çalıştırdı.

- `enqueue(A)`: Kuyruk dolu değil. `front == -1` olduğu için `front = 0` olur. `rear` -1 idi; $(-1+1) \bmod 4 = 0$ olur. A, 0 numaralı göze yerleşir. `front` ve `rear` aynı gözü gösterir.
- `enqueue(B)`: Kuyruk dolu değil, `front` -1 değil. $(0+1) \bmod 4 = 1$; `rear` 1 olur ve B oraya yerleşir.
- `enqueue(C)`, `enqueue(D)`: `rear` önce 2, sonra 3 olur.
- `enqueue(E)`: `front == rear + 1` sağlanmaz ($0 \neq 4$). Ama `front == 0` ve `rear == SIZE - 1 = 3` sağlanır. `isFull` doğru döner, `enqueue` 0 döndürür ve başarısız olur.
- `dequeue`: `front`taki A alınır. `front == rear` değildir ($0 \neq 3$). `front` $(0+1) \bmod 4 = 1$ olur. 0 numaralı göz teknik olarak boşalmıştır.
- `enqueue(E)`: `isFull` yanlış döner. $(3+1) \bmod 4 = 0$; `rear` başa döner ve E, 0 numaralı göze yerleşir.
- `enqueue(F)`: `front == rear + 1` ($1 = 0 + 1$) sağlanır. Kuyruk doludur, F eklenemez.

İşlem	Dizi (göz 0..3)	Kuyruk	`front`	`rear`	Sonuç
Başlangıç	- - - -	boş	-1	-1	
`enqueue(A)`	A - - -	A	0	0	başarılı
`enqueue(B), C, D`	A B C D	A B C D	0	3	başarılı
`enqueue(E)`	A B C D	A B C D	0	3	dolu, başarısız
`dequeue`	(A) B C D	B C D	1	3	A çıktı
`enqueue(E)`	E B C D	B C D E	1	0	başarılı
`enqueue(F)`	E B C D	B C D E	1	0	dolu, başarısız

Programın çıktısı tabloyla aynıdır:

```
enqueue(A)   ok=1  front= 0 rear= 0  kuyruk: A
enqueue(B)   ok=1  front= 0 rear= 1  kuyruk: A B
enqueue(C)   ok=1  front= 0 rear= 2  kuyruk: A B C
enqueue(D)   ok=1  front= 0 rear= 3  kuyruk: A B C D
enqueue(E)   ok=0  front= 0 rear= 3  kuyruk: A B C D
dequeue      ok=1  front= 1 rear= 3  kuyruk: B C D
enqueue(E)   ok=1  front= 1 rear= 0  kuyruk: B C D E
enqueue(F)   ok=0  front= 1 rear= 0  kuyruk: B C D E
```

7.5 `front == rear` Kontrolü Ne İçin?

Hoca `dequeue`'daki `front == rear` kontrolünün hangi senaryoda oluştuğunu sordu. Bir öğrenci elemanlar tek tek çıkarılıp kuyruk boşalınca `front`un -1'e çekilmesi gerektiğini anlattı. Hoca -1'e eşitlemenin gerçekten kuyruğun boş olduğunu anlamak için yapıldığını onayladı, ama slayttaki asıl durumu gösterdi:

Kuyruk boşken `enqueue(A)` yapılır. `front` -1'den 0'a çekilir. `rear` da $(-1+1) \bmod 4 = 0$ olur. İlk eleman eklendiğinde ikisi de 0'ı gösterir. Hemen ardından başka eleman eklenmeden `dequeue` yapılırsa A alınır ve `front == rear` olduğu için ikisi yeniden -1'e çekilir. Kuyruk eski haline döner.

Hoca bunu `enqueue`'daki `front == -1` kontrolüyle eşleştirdi: O kontrol yalnız ilk eleman eklenirken çalışır; bu kontrol de ilk elemandan sonra başka eleman eklenmeden `dequeue` yapıldığında çalışır. Genel olarak `front == rear` olduğu anda kuyrukta tek eleman vardır; o da çıkınca kuyruk boşalır. Program çıktısında da `enqueue(A)` sonrası `front = rear = 0`, `dequeue` sonrası `front = rear = -1` görülür.

7.6 Özet

- `initQueue`: `front = rear = -1`.
- `isEmpty`: `front == -1` ise 1 döner.
- `isFull`: `front` değer olarak `rear`dan bir büyükse ya da hiç eleman çıkmadan dizi dolmuşsa (`front == 0`, `rear == SIZE - 1`) 1 döner.
- `enqueue`: İlk eleman ekleniyorsa `front` 0 yapılır. `rear` da bu işlem sonunda 0 olur; ikisi aynı yeri gösterir.
- `dequeue`: İlk eleman eklendikten hemen sonra başka eleman eklenmeden çıkarılıyorsa (genel olarak son eleman çıkıyorsa) `front` ve `rear` -1'e döner.
- Modüler aritmetik, `front` ve `rear` dizinin sonuna geldiğinde yeniden başa dönebilmeleri için kullanılır.

  Ders sonrası not: İki doluluk koşulu tek bir ifadeyle de yazılabilir: `(rear + 1) % SIZE == front`. `front = 0`, `rear = SIZE - 1` iken $(SIZE - 1 + 1) \bmod SIZE = 0$ olur; `front = rear + 1` durumunu da aynı ifade kapsar.

8. Yığın ile Kuyruğun Karşılaştırılması

Özellik	Yığın	Kuyruk
Çalışma prensibi	LIFO	FIFO
Ekleme ve silme noktası	Tek uç	Bir uçtan ekleme (`rear`), öbür uçtan silme (`front`)
İşaretçi sayısı	1 (`top`)	2 (`front`, `rear`)
İşlemler	`push`, `pop`	`enqueue`, `dequeue`
Varyantlar	Farklı bir varyasyonu yok	Dairesel, öncelikli, çift uçlu kuyruk

Hoca kuyruğun bu anlamda yığına göre daha çeşitli olduğunu söyledi. Yığında rahatça yapılan işler kuyrukta dikkatli ve özenli yapılmazsa sorun çıkar. Bugünkü yapılarda iki işaretçinin birbirine göre konumunun ne kadar kritik olduğu ve ekstra kontrollere ihtiyaç duyulduğu görüldü. Kuyruk bu anlamda daha karmaşık bir yapıdır.

Soru: Yığın ve kuyruğun arkasında dizi kullanılıyorsa rastgele erişim de mümkün. Örneğin işlemcilerde yığın bir bellek alanı üzerindedir; yığın işaretçisi değiştirilerek yığının istenen bölgesine erişilir, değer yazılır, sonra işaretçi eski haline getirilir. Bunu neden tanımlamıyoruz?

Cevap: Dizi kullanıldığı için kâğıt üzerinde istenen göze erişilebilir gibi görünür. Ama bize verilen yığın gerçeklemesinde kurallar gereği yalnız `push` ve `pop` yapılabilir. Yığının en üstüne, kuyruğun da başına ve sonuna, işaretçiler üzerinden erişme hakkımız vardır. Özel olarak tanımlanmışsa olabilir, ama normalde böyle bir şey yoktur. Sorularda bu erişimin olmadığını varsayacağız. Yığının istenen yerine erişebiliyorsak yığından bahsetmeye gerek kalmaz; dizinin üzerine yığın gibi davranan fonksiyonlar yazarız. Rastgele erişim sırayı bozma ihtimali de doğurur; kuyruk için de aynısı geçerlidir. Örneğin öncelikli kuyrukta öncelikli bir eleman gelince kuyruğu yeniden düzenleyip elemanı tepeye yerleştirmek için ayrı fonksiyonlar tanımlanır; yığın ağacı (heap) yapısı bunun için vardır.

Bir başka öğrenci de soyut veri yapısı olduğu için nasıl kullanılacağının ve hangi metotların olacağının tanımda belirtildiğini, gerçeklemenin buna uygun yapılması gerektiğini ekledi. Hoca bunun bir kısıt olduğunu, teknik olarak belleğin o bölgesine adresle ulaşmanın bir engeli olmadığını söyledi.

9. İki Yığınla Kuyruk Gerçekleme

Hoca "stack and queue questions" başlığıyla bir soru sordu: İki yığın kullanarak bir kuyruk gerçeklemeniz istenseydi ne yapardınız?

Bir öğrenci hızlıca özetledi: Soldaki yığına A, sağdakine B diyelim. Yeni eleman her zaman A'ya `push` edilir. Eleman istendiğinde A'daki bütün elemanlar sırayla B'ye atılır ve son eleman çıkarılır. Sonra kalanlar aynı sırayla A'ya geri alınır.

Hoca bunu adım adım anlattı:

- **enqueue:** Teknik olarak bir push işlemidir. A yığınına basılır. 5, 6, 7 gelirse A'da alttan üste 5, 6, 7 olur.
- **dequeue:** Çıkarılacak eleman ilk konan elemandır, yani A'nın en altındadır. Ona ulaşmak için A'dan pop edilip B'ye push edilir. En son pop edilen eleman (5), kuyruktan çıkan elemandır. Sonra B'den pop edilip A'ya push edilerek 6 ve 7 eski yerlerine konur.
- Yeni bir enqueue gelirse eleman yine A'ya eklenir. Yeni bir dequeue gelirse aynı taşıma tekrarlanır.

Hoca bunun ilk akla gelen kaba kuvvet (brute force) çözüm olduğunu, iki yığını dengeli kullanan farklı bir çözüm de üretilebileceğini söyledi.

Bu çözüm C ile denendi. 5, 6, 7 eklendikten sonra dequeue 5'i verir; 8 eklendikten sonraki dequeue'lar sırayla 6, 7, 8'i verir. n elemanlı kuyrukta bir dequeue, A'dan n pop, B'ye $n-1$ push, B'den $n-1$ pop ve A'ya $n-1$ push yapar. Toplam $4n-3$ yığın işlemidir; $n=3$ için 9 işlem sayıldı. Yani enqueue $O(1)$, dequeue $O(n)$'dir.

> **Ek bilgi (derste anlatılmadı):** Hocanın andığı "iki yığını dengeli kullanma" çözümünde elemanlar B'den A'ya geri taşınmaz. dequeue geldiğinde B boş değilse doğrudan B'den pop edilir. B boşsa A'nın bütün elemanları bir kez B'ye aktarılır. Her eleman en fazla bir kez A'dan B'ye geçtiği için uzun bir işlem dizisinde işlem başına ortalama maliyet $O(1)$ olur (amortize çözümleme).

10. Kuyruk ile Yığın Gerçekleme

10.1 Soru ve İlk Öneri

Hoca bu kez tersini sordu: Kuyruk kullanarak yığın gerçeklemek istiyorsunuz. Tek bir kuyrukla yapabilir misiniz, birden fazla kuyruğa ihtiyacınız olur mu?

Bir öğrenci reardan hem ekleme hem çıkarma yapılırsa yığın elde edileceğini önerdi. Hoca adım adım gitti: push(A) gelince A kuyruğa enqueue ile konur. B, C, D de öyle. rear D'yi, front A'yı gösterir. Şimdi bir pop gelirse D çıkmalıdır. Öğrenci, soyut veri yapısının tanımı gereği reardan çıkarma yapılamayacağını, çıkarmanın yalnız fronttan olabileceğini kendisi fark etti.

10.2 İki Kuyrukla Çözüm

Bir başka öğrenci elemanları sona göndererek kuyruğu döndürmeyi önerdi. Hoca öneriyi tahtada izlerken önce kafasında oturtamadığını söyledi ve iki kuyrukla şu çözümü gösterdi:

- **push:** Doğrudan enqueue. A, B, C, D sırayla A kuyruğuna konur.
- **pop:** D'nin alınabilmesi için onun kuyruğun başına gelmesi gerekir. A kuyruğundan A dequeue edilip B kuyruğuna enqueue edilir. B ve C için de aynısı yapılır. Geriye kalan son eleman D, dequeue edilip dışarı verilir. Sonra A, B, C yeniden A kuyruğuna taşınır.
- Yeni bir pop gelirse aynı işlem tekrarlanır: A ve B öbür kuyruğa geçer, C dışarı verilir.

Öneriyi yapan öğrenci de kendi düşündüğünün bu olmadığını, enqueue'nun hep sona ekleyeceğini gözden kaçırdığını söyledi.

Hoca neden iki yapı gerektiğini şöyle açıkladı: push ve pop gerçeklenirken yalnız enqueue ve dequeue kullanma zorunluluğu vardır. front ve rear ile ekstra bir şey yapılamaz, kuyruğun formu bozulamaz. Bu yüzden bir yığın için iki kuyruk, bir kuyruk için iki yığın kullanılır; elemanlar hazır metotlarla birinden diğerine aktarılır. push tek bir enqueue ile $O(1)$ zamanda yapılır. pop için ise dequeue ve enqueue'lar yapılmak zorundadır ve maliyeti $O(n)$'dir. Hoca normalde bunu yapmanın çok akıllıca olmadığını, ama bu iki problemin literatürde sıkça geçtiğini ve göz atmanın faydalı olacağını söyledi.

C denemesinde A, B, C, D push edildikten sonra iki kuyruklu pop D'yi, ikinci pop C'yi verdi. n elemanlı yığında bir pop, $(n-1)$ dequeue + $(n-1)$ enqueue + 1 dequeue + $(n-1)$ dequeue + $(n-1)$ enqueue $= 4n-3$ kuyruk işlemidir; $n=4$ için 13 işlem sayıldı.

10.3 Tek Kuyrukla Çözüm

Bir öğrenci ikinci kuyruğa gerek olmadığını söyledi: `size` bilgisi tutulan dairesel kuyrukta her `pop`'ta `size - 1` kez `dequeue` ve hemen ardından aynı kuyruğa `enqueue` yapılır. A, B, C, D varken A `dequeue` edilip D'nin arkasına `enqueue` edilir; B ve C de öyle. `size - 1` döndürmeden sonra baştaki eleman D'dir; `dequeue` edilince `pop` biter. Sonraki `pop`'ta aynı işlem C'yi verir.

Hoca bunun mantıklı göründüğünü, atlanan bir şey olup olmadığına bakıp bir sonraki derste netleştireceğini söyledi.

Bir başka öğrenci kuyruğun boyutunun yetip yetmeyeceğini sordu: 4 gözlük dizide A, B, C, D doluysa A'yı yeniden `enqueue` etmeye yer olmayabilir. Hoca bunun haklı bir istisnai durum olduğunu söyledi ve dairesel kuyruk varsayımıyla birlikte inceledi: A `dequeue` edilince bir göz boşalır ve A, D'nin arkasına `enqueue` edilebilir. B için de aynısı yapılabilir. Böylece soru soran öğrencinin kafasındaki sorun da çözüldü.

> **Ders sonrası not:** Tek kuyruk çözümü doğrudur. C ile, kapasitesi 4 olan ve tamamen dolu bir dairesel kuyrukta denendi: Her `dequeue` bir göz açar, hemen ardından gelen `enqueue` o gözü kullanır, bu yüzden döndürme sırasında kuyruk taşmaz. A, B, C, D `push` edildikten sonra `pop`'lar D ve C'yi verdi; E `push` edildikten sonra `pop`'lar E, B, A'yı verdi. Bir `pop`, $(n-1)$ `dequeue` + $(n-1)$ `enqueue` + 1 `dequeue` $= 2n-1$ işlem yapar; maliyet yine $O(n)$'dir.

Hoca iki örneğin özünü tekrarladı: Birinde yığın fonksiyonlarıyla kuyruk, diğerinde kuyruk fonksiyonlarıyla yığın gerçeklendi. İkisi de beyin jimnastiği için faydalı örneklerdir.

Örnek Sorular

Bu derste hazır kare ve video yoktu. Aşağıdaki sorular derste sözlü sorulan ya da hocanın tahtaya yazıp sorduğu sorulardır.

Soru 1 (derste sözlü, 00:18:46): $A * B + C/D$ infix ifadesinin prefix karşılığı nedir?

Çözüm (derste verildi): `+ * A B / C D`. Önce $A * B$, sonra C/D yapılır, en son ikisi toplanır.

Soru 2 (derste sözlü, 00:20:32): $A+B*C$ ifadesini prefix'e çeviren kaba bir algoritma kurmak isteseydik nereden başlardık?

Çözüm (derste verildi): İfade ters çevrilir (`C * B + A`), postfix'e dönüştürülür (`C B * A +`), sonuç yeniden ters çevrilir (`+ A * B C`).

Soru 3 (derste sözlü, 00:25:55): Parantezli bir ifadede infix'ten prefix'e dönüşüm yapılırken önceki örneğe göre fark nedir?

Çözüm (derste verildi): Ters çevirmeden sonra parantezler de ters çevrilir; `(` yerine `)`, `)` yerine `(` yazılır. Ayrıca operatör karşılaştırmasında "eşit ya da büyük" yerine yalnız "büyük" durumu gözetilir.

Soru 4 (derste sözlü, 00:28:20): $A/B \wedge C+D*E-A*C$ ifadesinin prefix ve postfix karşılıkları nedir?

Çözüm (derste verildi, python ile doğrulandı): Prefix `- + / A ^ B C * D E * A C`, postfix `A B C ^ / D E * + A C * -`.

Soru 5 (derste sözlü, 00:31:17): $A \wedge B * C - D + E/F/(G+H)$ ifadesinin prefix ve postfix karşılıkları nedir?

> **Çözüm (derste verilmedi):** Öncelik ve soldan birleşme kurallarına göre ifade $(((A \wedge B) * C) - D) + ((E/F)/(G+H))$ biçiminde gruplanır. Python ile kesinleşen sonuçlar: Postfix `A B ^ C * D - E F / G H + / +`, prefix `+ - * ^ A B C D / / E F + G H`. İki ifade de rastgele değerlerde infix ifadeyle aynı sonucu verir.

Soru 6 (derste sözlü, 00:31:42): $A - B/(C * D \wedge E)$ ifadesinin prefix ve postfix karşılıkları nedir?

> **Çözüm (derste verilmedi):** İfade $A-(B/(C*(D \wedge E)))$ biçiminde gruplanır. Python ile kesinleşen sonuçlar: Postfix `A B C D E ^ * / -`, prefix `- A / B * C ^ D E`.

Soru 7 (derste sözlü, 00:33:33): İçinde 1, 2, 3, 4 olan bir yığını ek alan kullanmadan nasıl ters çevirirsiniz?

Çözüm (derste verildi): Yığın bağlı liste ile gerçeklenir. Liste `NULL` görülene kadar dolaşılır, her düğümün bağlantısı ters yöne çevrilir, en sonda `head` eski en alttaki düğüme atanır. $O(n)$ zaman, $O(1)$ ek alan (Bölüm 3.3'teki kod derlendi).

Soru 8 (derste sözlü, 01:07:05): `front` ve `rear` -1 iken `enqueue(A)` yapılırsa ikisi nereyi gösterir? Bu durumda `dequeue` A'yı çıkarabilir mi?

Çözüm (derste verildi, C ile doğrulandı): `rear` 0'ı gösterir, `front` -1'de kalır. `isEmpty` `front == -1` kontrolü yaptığı için kuyruk boş görünür ve A çıkarılamaz.

Soru 9 (derste sözlü, 01:16:27): Bu sorunu nasıl çözeriz? Başlangıç noktaları nerede olmalıdır?

Çözüm (derste verildi, C ile doğrulandı): `front = rear = -1`, `q[++rear]`, `q[++front]` ve boşluk için `front == rear`; ya da ayrı bir `size` sayacı tutup kuyruk boşalınca işaretçileri başa almak. `front`u ilk `enqueue`'da 0'a çekip `dequeue`'da `++front` kullanmak A yerine B'yi çıkarır.

Soru 10 (derste sözlü, 01:30:35): Dairesel kuyrukta `front` ve `rear` hangi konumda olursa kuyruk dolu deriz?

Çözüm (derste verildi): Arkalı önlü olduklarında, yani `front == rear + 1` iken; ya da başlangıçtan beri hiç eleman çıkmadan dizi dolduğunda (`front == 0`, `rear == SIZE - 1`).

Soru 11 (derste sözlü, 01:32:39): `front == 0 && rear == SIZE - 1` durumu ne zaman oluşur?

Çözüm (derste verildi): Elemanlar hiç durmadan gelmiş, hiç `dequeue` yapılmamıştır. `front` başta kalmış, `rear` dizinin sonuna kadar gitmiştir.

Soru 12 (derste sözlü, 01:34:37): `enqueue`'daki `if (front == -1)` kontrolü hangi durum için yapılır?

Çözüm (derste verildi): Kuyruğa ilk kez eleman eklenirken `frontu` 0'a çekmek için.

Soru 13 (derste sözlü, 01:40:07): `dequeue`'daki `front == rear` kontrolü hangi senaryoda oluşur?

Çözüm (derste verildi): Kuyrukta tek eleman varken. Örneğin boş kuyruğa A eklenince `front` ve `rear` 0'ı gösterir; hemen `dequeue` yapılırsa A çıkar ve ikisi yeniden -1'e çekilir.

Soru 14 (derste sözlü, 02:00:45): İki yığın kullanarak bir kuyruk nasıl gerçeklenir?

Çözüm (derste verildi, C ile doğrulandı): `enqueue` A'ya `push` edilir. `dequeue`'da A boşaltılıp B'ye aktarılır, en alttaki (ilk gelen) eleman verilir, kalanlar A'ya geri taşınır. `dequeue` $4n - 3$ yığın işlemi, yani $O(n)$.

Soru 15 (derste sözlü, 02:08:31): Kuyruk kullanarak yığın gerçeklemek isterseniz tek kuyruk yeter mi, birden fazla kuyruk gerekir mi?

Çözüm (derste iki kuyruklu sürüm verildi; tek kuyruk sürümü hoca tarafından sonraki derse bırakıldı): İki kuyrukla, son eleman hariç hepsi öbür kuyruğa aktarılır, kalan eleman verilir, kalanlar geri taşınır ($4n - 3$ işlem).

> **Çözüm (derste verilmedi):** Tek kuyruk yeterlidir. `pop`'ta `size - 1` eleman `dequeue` edilip aynı kuyruğa `enqueue` edilir, sonra baştaki eleman `dequeue` edilir. Tamamen dolu, 4 gözlük dairesel kuyrukta C ile denendi ve doğru sırayla D, C, E, B, A verdi. Maliyet $2n - 1$ işlem, yani $O(n)$'dir.

Dersin Toparlaması

Bu derste önce yığının kullanım alanları ve sınırları toparlandı. Ardından yığınla çözülen iki problem işlendi. Infix'ten prefix'e dönüşüm, ters çevir, postfix'e dönüştür, yeniden ters çevir adımlarıyla yapılır; parantezlerin ters çevrilmesi ve öncelik karşılaştırmasında yalnız "büyük" durumunun gözetilmesi bu dönüşümün inceliklerdir. Ek alan kullanmadan ters çevirmede ise bağlı liste ile kurulmuş yığının bağlantıları ters yöne çevrildi ve alan ile zaman arasındaki takas vurgulandı.

Dersin ikinci yarısında kuyruk yapısı FIFO mantığı, `front` ve `rear` işaretçileri, `enqueue` ve `dequeue` işlemleriyle tanıtıldı. Dizi ile kurulan normal kuyrukta başlangıç değerleri ve artırma sırasının uyumsuz olmasının nasıl hatalara yol açtığı birlikte incelendi. Alanı verimli kullanan dairesel kuyrukta modüler aritmetik, iki doluluk koşulu ve son eleman çıkınca işaretçilerin -1'e çekilmesi görüldü. Son olarak yığın ve kuyruk karşılaştırıldı, iki yığınla kuyruk ve kuyrukla yığın gerçekleme soruları çözüldü. Hoca önümüzdeki hafta yığın ve kuyrukla ilgili birkaç örnek daha çözüleceğini, tek kuyruk önerisinin netleştirileceğini ve ardından ağaçlara geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.